PIMPRI CHINCHWAD EDUCATION TRUST's

PIMPRI CHINCHWAD COLLEGE OF ENGINEERING

(An Autonomous Institute)

DATABASE MANAGEMENT SYSTEM LABORATORY

NAME -	Viraj Pathare
PRN -	124B1F149
CLASS -	SY - IT
DIV -	A

ASSIGNMENT NO. 4 :- Procedure and Function.

Problem Statement: Write code to implement Procedure and Function.

A startup's streaming platform database was partially damaged — records are inconsistent, missing, and duplicated. As a **Database Detective**, this report implements **Stored Procedures** and **Functions** to automate, encapsulate, and investigate the database logic based on the platform's ER diagram.

The ER diagram defines six core entities for the streaming platform. Below is the schema used throughout this assignment.

```
-- 1. CONTENT
CREATE TABLE Content (
content_id INT PRIMARY KEY AUTO_INCREMENT, title
VARCHAR(200) NOT NULL,
genre VARCHAR(100),
language VARCHAR(50), duration
INT, -- minutes release_date DATE,
age_rating VARCHAR(10)
);

-- 2. SUBSCRIPTION_PLAN
CREATE TABLE Subscription_Plan ( plan_id INT
PRIMARY KEY AUTO_INCREMENT,
plan_name VARCHAR(100) NOT NULL,
price DECIMAL(8,2) NOT NULL CHECK (price >= 0),
video_quality VARCHAR(20) NOT NULL DEFAULT 'HD', max_screens INT
NOT NULL DEFAULT 1
);

-- 3. USER
CREATE TABLE User (
user_id INT PRIMARY KEY AUTO_INCREMENT, name
VARCHAR(100) NOT NULL,
email VARCHAR(150) NOT NULL UNIQUE, phone
VARCHAR(15),
password VARCHAR(255) NOT NULL,
registration_date DATE NOT NULL DEFAULT (CURDATE())
);
```

```
-- 4. SUBSCRIPTION (bridges User & Subscription_Plan) CREATE TABLE
Subscription (
subscription_id INT PRIMARY KEY AUTO_INCREMENT, user_id INT NOT
NULL,
plan_id INT NOT NULL, start_date
DATE NOT NULL, end_date DATE
NOT NULL,
status ENUM('Active','Expired','Cancelled') NOT NULL DEFAULT 'Active', FOREIGN KEY
(user_id) REFERENCES User(user_id) ON DELETE CASCADE, FOREIGN KEY (plan_id)
REFERENCES Subscription_Plan(plan_id)
);

-- 5. PAYMENT
CREATE TABLE Payment (
payment_id INT PRIMARY KEY AUTO_INCREMENT,
subscription_id INT NOT NULL,
payment_date DATE NOT NULL DEFAULT (CURDATE()), amount
DECIMAL(8,2) NOT NULL CHECK (amount > 0),
payment_method ENUM('Credit Card','Debit Card','UPI','Net Banking') NOT NULL, payment_status
ENUM('Success','Failed','Refunded') NOT NULL DEFAULT 'Success', FOREIGN KEY (subscription_id) REFERENCES
Subscription(subscription_id) );

-- 6. WATCH_HISTORY
CREATETABLEWatch_History (
history_id INT PRIMARY KEY AUTO_INCREMENT,
user_id INT NOT NULL, content_id
INT NOT NULL,
watch_date DATE NOT NULL DEFAULT (CURDATE()),
watch_id INT,
FOREIGN KEY (user_id)REFERENCES User(user_id) ON DELETECASCADE,
FOREIGN KEY (content_id) REFERENCESContent(content_id)
);
```

Stored Procedures encapsulate reusable SQL logic that can accept parameters, perform DML operations, and return result sets. All procedures below are designed around the streaming platform schema.

Procedure 1 — Register New User

Inserts a new user record into the **User** table. Validates that the email is not already registered before inserting. Raises a custom error if a duplicate email is detected.

```
DELIMITER$$

CREATE PROCEDURE sp_RegisterUser ( IN
p_name VARCHAR(100),
INp_email VARCHAR(150), IN
p_phone VARCHAR(15),
INp_passwordVARCHAR(255), OUT
p_user_id INT
)
BEGIN
-- Check for duplicate email
IF EXISTS (SELECT 1 FROM User WHERE email = p_email) THEN SIGNAL
SQLSTATE '45000'
SET MESSAGE_TEXT = 'Email already registered.'; ELSE
```

```
INSERT INTO User (name, email, phone, password) VALUES (p_name,
p_email, p_phone, p_password);

SET p_user_id = LAST_INSERT_ID(); END IF;
END$$

DELIMITER;

-- Usage:
CALL sp_RegisterUser(
'Arjun Verma', 'arjun@stream.com', '9876543210', 'hashed_pwd_123',
@new_id
);
SELECT @new_id AS newly_created_user_id;
```

Sample Output:

13

Procedure 2 — Subscribe User to a Plan

Creates a new subscription for a user by inserting into the **Subscription** table and immediately recording a corresponding **Payment**. Automatically calculates the end date as 30 days from the start date.

```
DELIMITER$$

CREATE PROCEDURE sp_SubscribeUser ( IN
p_user_id INT,
IN p_plan_id INT,
IN p_start_date DATE,
IN p_payment_method ENUM('Credit Card','Debit Card','UPI','Net Banking') )
BEGIN
DECLARE v_end_date DATE;
DECLAREv_price DECIMAL(8,2);
DECLAREv_sub_id INT;

-- Calculate end date (30-day cycle)
SETv_end_date = DATE_ADD(p_start_date, INTERVAL30 DAY);

-- Fetch plan price SELECT price
INTO v_price FROM
Subscription_Plan WHERE plan_id =
p_plan_id;

-- Insertsubscription record
INSERT INTO Subscription (user_id, plan_id, start_date, end_date, status) VALUES (p_user_id,
p_plan_id, p_start_date, v_end_date, 'Active');
SET v_sub_id = LAST_INSERT_ID();

-- Record corresponding payment
INSERT INTO Payment (subscription_id, payment_date, amount, payment_method) VALUES (v_sub_id, p_start_date, v_price,
p_payment_method);

SELECT v_sub_id AS subscription_id, v_end_date AS end_date, v_price AS
```

```
amount_charged;
END$$

DELIMITER;

-- Usage:
CALLsp_SubscribeUser(3, 2, '2025-07-01', 'UPI');
```

Sample Output:

7	2025-07-31	299.00

Procedure 3 — Record Watch History

Logs a content viewing event for a user into **Watch_History**. Validates that the user holds an **Active** subscription before allowing the record to be inserted. Prevents unauthorised streaming without a valid plan.

```
DELIMITER$$

CREATE PROCEDURE sp_RecordWatchHistory ( IN
p_user_id INT,
INp_content_id INT, IN
p_watch_date DATE
)
BEGIN
-- Allow only active subscribers to watch IF NOT EXISTS (
SELECT 1 FROM Subscription
WHERE user_id = p_user_id AND status = 'Active' AND p_watch_date
BETWEEN start_date AND end_date
) THEN
SIGNALSQLSTATE'45000' SET
MESSAGE_TEXT =
'No active subscription. Cannot record watch history.'; ELSE
INSERT INTO Watch_History (user_id, content_id, watch_date) VALUES (p_user_id,
p_content_id, p_watch_date);

SELECT LAST_INSERT_ID() AS history_id, 'Watch
recorded successfully.' AS message; END IF;
END$$

DELIMITER;

-- Usage:
CALL sp_RecordWatchHistory(1, 5, '2025-07-10');
```

Sample Output:

24	Watch recorded successfully.

Procedure 4 — Cancel Subscription & Initiate Refund

Cancels a subscription by updating its status to **Cancelled** and marks the latest associated payment as **Refunded**. Simulates a

real-world subscription cancellation workflow in a single atomic operation.

```
DELIMITER$$

CREATE PROCEDURE sp_CancelSubscription ( IN
p_subscription_id INT,
OUT p_result VARCHAR(100)
)
BEGIN
DECLAREv_status VARCHAR(20);

SELECT statusINTO v_status FROM
Subscription
WHERE subscription_id = p_subscription_id;

IF v_status IS NULL THEN
SET p_result = 'Subscription not found.'; ELSEIF v_status =
'Cancelled' THEN
SET p_result = 'Already cancelled.'; ELSE
--Cancelsubscription UPDATE
Subscription
SET status = 'Cancelled'
WHERE subscription_id = p_subscription_id;

-- Mark latest payment as Refunded UPDATE Payment
SET payment_status = 'Refunded'
WHERE subscription_id = p_subscription_id ORDER BY
payment_date DESC
LIMIT 1;

SET p_result = 'Subscription cancelled and refund initiated.'; END IF;
END$$

DELIMITER;

-- Usage:
CALLsp_CancelSubscription(3, @msg); SELECT @msg
AS result;
```

Sample Output:

Subscription cancelled and refund initiated.

Procedure 5 — Get User Subscription Report

Generates a comprehensive subscription report for a given user, joining **User**, **Subscription**, **Subscription_Plan**, and **Payment** to provide full visibility of the user's account status.

```
DELIMITER$$

CREATE PROCEDURE sp_UserSubscriptionReport ( IN p_user_id
INT
) BEGIN
SELECT
u.user_id, u.name,
u.email,
sp.plan_name,
```

```
sp.price ASmonthly_price,
sp.video_quality, sp.max_screens,
s.start_date,
s.end_date,
s.statusASsubscription_status, py.payment_date,
py.amountASamount_paid,
py.payment_method,
py.payment_status
FROM User u
JOIN Subscription s ON u.user_id = s.user_id
JOIN Subscription_Plan sp ON s.plan_id = sp.plan_id
LEFT JOIN Payment py ON s.subscription_id = py.subscription_id WHERE u.user_id =
p_user_id
ORDER BY s.start_date DESC; END$$

DELIMITER;

-- Usage:
CALL sp_UserSubscriptionReport(1);
```

Sample Output:

1	Arjun Verma	Premium	4K Ultra HD	Active	649.00

User-Defined Functions (UDFs) return a single computed value and can be embedded directly inside SELECT, WHERE, or HAVING clauses. The functions below implement key business rules of the streaming platform.

Function 1 — fn_IsSubscriptionActive()

Returns 1 if the specified user has at least one **Active** subscription on the given date, otherwise returns 0. Used as a guard before allowing content playback.

```
DELIMITER$$

CREATE FUNCTION fn_IsSubscriptionActive ( p_user_id
INT,
p_check_dateDATE
)
RETURNSTINYINT(1)
DETERMINISTIC
READS SQL DATA
BEGIN
DECLARE v_count INT DEFAULT 0;

SELECT COUNT(*) INTO v_count
FROM Subscription
WHERE user_id = p_user_id AND
status = 'Active'
AND p_check_date BETWEEN start_date AND end_date;

RETURN IF(v_count > 0, 1, 0);
END$$

DELIMITER;
```

```
-- Usage:
SELECT user_id, name,
fn_IsSubscriptionActive(user_id, CURDATE()) AS is_active FROM User;
```

Sample Output:

1	Arjun Verma	1
2	Priya Nair	1
3	Rohan Mehta	0
4	Sneha Iyer	1

Function 2 — fn_GetTotalRevenue()

Calculates and returns the total revenue collected from successful payments for a specific subscription plan. Useful for financial reporting and plan performance analysis.

```
DELIMITER$$
CREATE FUNCTION fn_GetTotalRevenue ( p_plan_id INT
)
RETURNSDECIMAL(12,2)
DETERMINISTIC
READS SQLDATA
BEGIN
DECLAREv_revenue DECIMAL(12,2) DEFAULT0.00;
SELECT COALESCE(SUM(py.amount), 0.00) INTO v_revenue FROM
Payment py
JOIN Subscription s ON py.subscription_id = s.subscription_id WHERE s.plan_id =
p_plan_id
AND py.payment_status = 'Success';

RETURNv_revenue; END$$

DELIMITER;

-- Usage:
SELECT plan_id, plan_name, fn_GetTotalRevenue(plan_id) AS
total_revenue FROM Subscription_Plan
ORDERBY total_revenue DESC;
```

Sample Output:

3	Premium	19500.00
2	Standard	11700.00
1	Basic	4199.00

Function 3 — fn_GetUserWatchCount()

Returns the total number of content items watched by a specific user. Enables quick user engagement scoring and can be used to identify power users versus inactive accounts.

```
DELIMITER$$

CREATE FUNCTION fn_GetUserWatchCount ( p_user_id
INT
)
RETURNS INT
DETERMINISTIC
READS SQLDATA
BEGIN
DECLARE v_count INT DEFAULT 0;

SELECT COUNT(*) INTO v_count
FROM Watch_History
WHERE user_id = p_user_id; RETURN
v_count;
END$$

DELIMITER;

-- Usage:
SELECT u.user_id, u.name, fn_GetUserWatchCount(u.user_id) AS
total_watched FROM User u
ORDERBY total_watched DESC;
```

Sample Output:

1	Arjun Verma	14
5	Karan Patel	11
2	Priya Nair	9
3	Rohan Mehta	3

Function 4 — fn_GetMostWatchedGenre()

Determines the most-watched content genre for a given user by joining **Watch_History** with **Content**, grouping by genre, and returning the top result. Enables personalised content recommendations.

```
DELIMITER$$

CREATE FUNCTION fn_GetMostWatchedGenre ( p_user_id
INT
)
RETURNSVARCHAR(100)
DETERMINISTIC
READS SQLDATA
BEGIN
DECLAREv_genre VARCHAR(100) DEFAULT'None';

SELECT c.genre INTO v_genre FROM
Watch_History wh
JOIN Content c ON wh.content_id = c.content_id WHERE wh.user_id =
p_user_id
AND c.genre IS NOT NULL GROUP
BY c.genre
```

```
ORDER BY COUNT(*) DESC
LIMIT 1;

RETURNCOALESCE(v_genre, 'None'); END$$

DELIMITER;

-- Usage:
SELECT u.user_id, u.name, fn_GetMostWatchedGenre(u.user_id) AS
favourite_genre FROM User u;
```

Sample Output:

1	Arjun Verma	Thriller
2	Priya Nair	Romance
5	Karan Patel	Action

Function 5 — fn_DaysUntilExpiry()

Computes the number of days remaining until a user's active subscription expires. Returns **-1** if no active subscription exists. Used to trigger renewal reminders before expiry.

```
DELIMITER$$

CREATE FUNCTION fn_DaysUntilExpiry ( p_user_id INT
)
RETURNS INT
DETERMINISTIC
READS SQLDATA
BEGIN
DECLARE v_daysINT DEFAULT -1;

SELECT DATEDIFF(end_date, CURDATE()) INTO v_days FROM
Subscription
WHERE user_id = p_user_id AND
status = 'Active'
AND CURDATE() BETWEEN start_date AND end_date ORDER BY
end_date DESC
LIMIT 1;

RETURNCOALESCE(v_days,-1);
END$$

DELIMITER;

-- Usage (renewal alert — expires within 7 days):
SELECT u.user_id, u.name, u.email,
fn_DaysUntilExpiry(u.user_id) AS days_left FROM User u
WHEREfn_DaysUntilExpiry(u.user_id)BETWEEN0 AND7;
```

Sample Output:

2	Priya Nair	priya@stream.com	4

6	Divya Reddy	divya@stream.com	2

Procedures and functions work together seamlessly. The query below demonstrates a single SELECT that embeds three UDFs alongside data from the User table — producing a full user dashboard in one shot.

```
-- Full user engagement dashboard SELECT
u.user_id, u.name,
u.email,
fn_IsSubscriptionActive(u.user_id,CURDATE()) AS is_active, fn_DaysUntilExpiry(u.user_id) AS
days_left, fn_GetUserWatchCount(u.user_id) AS total_watched,
fn_GetMostWatchedGenre(u.user_id) AS fav_genre
FROM User u
ORDERBY total_watched DESC;
```

Sample Output:

1	Arjun Verma	1	18	14	Thriller
5	Karan Patel	1	25	11	Action
2	Priya Nair	1	4	9	Romance
3	Rohan Mehta	0	-1	3	Drama

Revenue Summary Using fn_GetTotalRevenue()

```
SELECT
sp.plan_id,
sp.plan_name,
sp.price AS monthly_price, COUNT(s.subscription_id) AS
total_subscribers, fn_GetTotalRevenue(sp.plan_id) AS total_revenue
FROM Subscription_Plan sp
LEFT JOIN Subscription s ON sp.plan_id = s.plan_id GROUP BY sp.plan_id,
sp.plan_name, sp.price
ORDERBY total_revenue DESC;
```

Sample Output:

3	Premium	649.00	30	19500.00
2	Standard	399.00	29	11700.00
1	Basic	199.00	21	4199.00

1	**Missing Payment Validation**	Subscription records were being marked Active without a corresponding successful Payment entry.sp_SubscribeUser now atomically inserts both records, closing the gap.

2	**Unauthorised Content Access**	Watch_History entries existed for users whose subscriptions had expired. sp_RecordWatchHistory now validates an active subscription before inserting.
3	**Cancelled Order Still Charged**	A subscription with status Cancelled still had a payment marked Success. sp_CancelSubscription now marks the latest payment as Refunded at cancellation.
4	**Duplicate User Registrations**	Two user accounts shared the same email. sp_RegisterUser raises an error on duplicate email, preventing such entries at the database layer.
5	**Inactive Subscribers Not Identified**	No mechanism existed to flag subscriptions about to

		expire. fn_DaysUntilExpiry enables proactive renewal reminders.
6	**No Revenue Visibility per Plan**	Finance had no quick way to query plan-level revenue. fn_GetTotalRevenue provides this in a single embedded function call.

Most Valuable Procedure

sp_SubscribeUser provided the highest operational value. It atomically creates both the Subscription and Payment records in a single call, ensuring referential integrity. Without this procedure, application-layer bugs could create subscriptions with no payment record — causing revenue leakage. The procedure also auto-computes end_date, removing manual calculation errors.

Most Valuable Function

fn_IsSubscriptionActive() is the most critical function as it acts as a universal access-control gate. It can be embedded in any query or procedure to enforce the rule: "only paying subscribers can watch content." This prevents unauthorised access at the database layer rather than relying solely on application logic.

Recommendation

Add a **BEFORE INSERT trigger** on Watch_History that calls fn_IsSubscriptionActive() and rejects the insert if it returns 0. This creates a permanent, application-independent enforcement layer. Additionally, introduce a scheduled event (MySQL Event Scheduler) to run nightly and auto-update subscriptions whose end_date has passed to status 'Expired', keeping the Subscription table always consistent.

Bonus 1 — Procedure with Cursor: Top Watchers Report

Uses a cursor to iterate through all users and builds a summary of their watch count and favourite genre, inserting results into a temporary report table.

```
DELIMITER$$

CREATE PROCEDURE sp_GenerateEngagementReport () BEGIN
DECLARE done INT DEFAULT 0;
DECLARE v_uid INT;
DECLARE v_name VARCHAR(100);
DECLARE v_watches INT; DECLARE
```

```
v_genre VARCHAR(100);

DECLARE cur CURSOR FOR
SELECT user_id, name FROM User;

DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;

-- Temp table for the report
CREATE TEMPORARY TABLE IF NOT EXISTS tmp_report (
user_id INT,
name VARCHAR(100),
watch_count INT, fav_genre
VARCHAR(100)
);
OPEN cur; fetch_loop:
LOOP
FETCH cur INTO v_uid, v_name;
IF done = 1 THEN LEAVE fetch_loop; END IF;

SET v_watches = fn_GetUserWatchCount(v_uid); SET v_genre =
fn_GetMostWatchedGenre(v_uid);

INSERT INTO tmp_report VALUES (v_uid, v_name, v_watches, v_genre); END LOOP;
CLOSE cur;

SELECT * FROM tmp_report ORDER BY watch_count DESC; DROP
TEMPORARY TABLE tmp_report;
END$$

DELIMITER;

-- Usage:
CALL sp_GenerateEngagementReport();
```

Sample Output:

1	Arjun Verma	14	Thriller

5	Karan Patel	11	Action
2	Priya Nair	9	Romance

Bonus 2 — Function: fn_GetSubscriptionLabel()

Returns a human-readable status label for a user's subscription — "Active", "Expiring Soon" (within 7 days), "Expired", or "No Subscription". Useful in dashboards and notification systems.

```
DELIMITER$$

CREATE FUNCTION fn_GetSubscriptionLabel ( p_user_id
INT
```

```
)
RETURNSVARCHAR(30)
DETERMINISTIC
READS SQL DATA
BEGIN
DECLARE v_daysINT;
SET v_days = fn_DaysUntilExpiry(p_user_id);

RETURN CASE
WHEN v_days = -1 THEN 'No Subscription'
WHEN v_days BETWEEN 0 AND 7 THEN 'Expiring Soon' ELSE
'Active'
END;
END$$

DELIMITER;

-- Usage:
SELECT user_id, name, fn_GetSubscriptionLabel(user_id) AS
status_label FROM User;
```

Sample Output:

1	Arjun Verma	Active
2	Priya Nair	Expiring Soon
3	Rohan Mehta	No Subscription

Bonus 3 — Trigger: Prevent Watch Without Active Subscription

A BEFORE INSERT trigger on Watch_History that calls fn_IsSubscriptionActive() and blocks the insert if the user has no valid subscription on the watch date. This enforces the access rule permanently at the database layer.

```
DELIMITER$$

CREATETRIGGERtrg_ValidateWatchAccess BEFORE
INSERT ON Watch_History
FOREACHROW
BEGIN
IF fn_IsSubscriptionActive(NEW.user_id, NEW.watch_date) = 0 THEN SIGNAL SQLSTATE
'45000'
SET MESSAGE_TEXT =
'Access denied: No active subscription on this date.'; END IF;
END$$

DELIMITER;
```

The trigger fires BEFORE every INSERT on Watch_History. If the user has no active subscription on the watch_date, the insert is rejected with a descriptive error — permanently closing the unauthorised access gap identified in Part D.